

sml-project-2303a52389

November 18, 2024

```
[155]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
[156]: df = pd.read_csv('/content/Sml_data.csv')
df
```

```
[156]:
```

	sex	age	height	weight	waistline	sight_left	sight_right	\
0	Male	35	170	75	90.0	1.0	1.0	
1	Male	30	180	80	89.0	0.9	1.2	
2	Male	40	165	75	91.0	1.2	1.5	
3	Male	50	175	80	91.0	1.5	1.2	
4	Male	50	165	60	80.0	1.0	1.2	
...	...	...	...	...	...	...	...	
3344	Male	70	165	65	86.0	1.2	1.2	
3345	Male	45	160	65	87.0	0.7	0.7	
3346	Female	60	165	60	92.0	0.7	0.9	
3347	Male	55	175	75	80.0	0.9	1.0	
3348	Male	65	160	55	83.0	0.3	0.4	

	hear_left	hear_right	SBP	...	LDL_chole	triglyceride	hemoglobin	\
0	1	1	120	...	126	92	17.1	
1	1	1	130	...	148	121	15.8	
2	1	1	120	...	74	104	15.8	
3	1	1	145	...	104	106	17.6	
4	1	1	138	...	117	104	13.8	
...	...	...	...	...	...	...	...	
3344	1	1	132	...	116	100	14.0	
3345	1	1	138	...	71	96	16.1	
3346	1	1	138	...	98	140	14.8	
3347	1	1	131	...	138	59	15.0	
3348	1	1	110	...	84	78	13.9	

	urine_protein	serum_creatinine	SGOT_AST	SGOT_ALT	gamma_GTP	\
0	1	1.0	21	35	40	
1	1	0.9	20	36	27	
2	1	0.9	47	32	68	

3	1	1.1	29	34	18
4	1	0.8	19	12	25
...	...	...	...	...	...
3344	1	0.9	23	21	16
3345	1	0.9	33	25	68
3346	1	0.6	24	19	8
3347	1	0.8	15	31	33
3348	1	0.6	20	19	21

	SMK_stat_type_cd	DRK_YN
0	1	Y
1	3	N
2	1	N
3	1	N
4	1	N
...	...	...
3344	2	Y
3345	1	Y
3346	1	N
3347	2	Y
3348	3	N

[3349 rows x 24 columns]

```
[157]: df.isnull()
```

```
[157]:
```

	sex	age	height	weight	waistline	sight_left	sight_right	\
0	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	
3	False	False	False	False	False	False	False	
4	False	False	False	False	False	False	False	
...	...	...	...	...	...	...	...	
3344	False	False	False	False	False	False	False	
3345	False	False	False	False	False	False	False	
3346	False	False	False	False	False	False	False	
3347	False	False	False	False	False	False	False	
3348	False	False	False	False	False	False	False	

	hear_left	hear_right	SBP	...	LDL_chole	triglyceride	hemoglobin	\
0	False	False	False	...	False	False	False	
1	False	False	False	...	False	False	False	
2	False	False	False	...	False	False	False	
3	False	False	False	...	False	False	False	
4	False	False	False	...	False	False	False	
...	...	...	...	...	...	...	...	
3344	False	False	False	...	False	False	False	

3345	False	False	False	...	False	False	False
3346	False	False	False	...	False	False	False
3347	False	False	False	...	False	False	False
3348	False	False	False	...	False	False	False

	urine_protein	serum_creatinine	SGOT_AST	SGOT_ALT	gamma_GTP	\
0	False	False	False	False	False	
1	False	False	False	False	False	
2	False	False	False	False	False	
3	False	False	False	False	False	
4	False	False	False	False	False	
...	...	...	...	...	...	
3344	False	False	False	False	False	
3345	False	False	False	False	False	
3346	False	False	False	False	False	
3347	False	False	False	False	False	
3348	False	False	False	False	False	

	SMK_stat_type_cd	DRK_YN
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
3344	False	False
3345	False	False
3346	False	False
3347	False	False
3348	False	False

[3349 rows x 24 columns]

1 Changing String to Integer Datatype

```
[158]: df['DRK_YN']=df['DRK_YN'].replace({'Y':1,'N':0})
df['sex'] = df['sex'].replace({'Male':1,'Female':0})
df
```

```
<ipython-input-158-b0408db997b3>:1: FutureWarning: Downcasting behavior in
`replace` is deprecated and will be removed in a future version. To retain the
old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to
the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
```

```
df['DRK_YN']=df['DRK_YN'].replace({'Y':1,'N':0})
```

```
<ipython-input-158-b0408db997b3>:2: FutureWarning: Downcasting behavior in
`replace` is deprecated and will be removed in a future version. To retain the
```

old behavior, explicitly call ``result.infer_objects(copy=False)``. To opt-in to the future behavior, set ``pd.set_option('future.no_silent_downcasting', True)``

```
df['sex'] = df['sex'].replace({'Male':1,'Female':0})
```

```
[158]:
```

	sex	age	height	weight	waistline	sight_left	sight_right	hear_left	\
0	1	35	170	75	90.0	1.0	1.0	1	
1	1	30	180	80	89.0	0.9	1.2	1	
2	1	40	165	75	91.0	1.2	1.5	1	
3	1	50	175	80	91.0	1.5	1.2	1	
4	1	50	165	60	80.0	1.0	1.2	1	
...	...	...	...	...	...	...	...	...	
3344	1	70	165	65	86.0	1.2	1.2	1	
3345	1	45	160	65	87.0	0.7	0.7	1	
3346	0	60	165	60	92.0	0.7	0.9	1	
3347	1	55	175	75	80.0	0.9	1.0	1	
3348	1	65	160	55	83.0	0.3	0.4	1	

	hear_right	SBP	...	LDL_chole	triglyceride	hemoglobin	\
0	1	120	...	126	92	17.1	
1	1	130	...	148	121	15.8	
2	1	120	...	74	104	15.8	
3	1	145	...	104	106	17.6	
4	1	138	...	117	104	13.8	
...	...	...	...	...	...	...	
3344	1	132	...	116	100	14.0	
3345	1	138	...	71	96	16.1	
3346	1	138	...	98	140	14.8	
3347	1	131	...	138	59	15.0	
3348	1	110	...	84	78	13.9	

	urine_protein	serum_creatinine	SGOT_AST	SGOT_ALT	gamma_GTP	\
0	1	1.0	21	35	40	
1	1	0.9	20	36	27	
2	1	0.9	47	32	68	
3	1	1.1	29	34	18	
4	1	0.8	19	12	25	
...	...	...	...	...	...	
3344	1	0.9	23	21	16	
3345	1	0.9	33	25	68	
3346	1	0.6	24	19	8	
3347	1	0.8	15	31	33	
3348	1	0.6	20	19	21	

	SMK_stat_type_cd	DRK_YN
0	1	1
1	3	0
2	1	0

3	1	0
4	1	0
...	...	...
3344	2	1
3345	1	1
3346	1	0
3347	2	1
3348	3	0

[3349 rows x 24 columns]

```
[159]: df.dtypes
```

```
[159]: sex                int64
age                int64
height            int64
weight            int64
waistline          float64
sight_left         float64
sight_right        float64
hear_left          int64
hear_right         int64
SBP                int64
DBP                int64
BLDS               int64
tot_chole           int64
HDL_chole           int64
LDL_chole           int64
triglyceride        int64
hemoglobin          float64
urine_protein        int64
serum_creatinine     float64
SGOT_AST            int64
SGOT_ALT            int64
gamma_GTP           int64
SMK_stat_type_cd     int64
DRK_YN              int64
dtype: object
```

```
[160]: target=df['DRK_YN']
target
```

```
[160]: 0      1
1      0
2      0
3      0
4      0
```

```

..
3344    1
3345    1
3346    0
3347    1
3348    0
Name: DRK_YN, Length: 3349, dtype: int64

```

```

[161]: features=df.drop(['DRK_YN'],axis=1)
features

```

```

[161]:
    sex  age  height  weight  waistline  sight_left  sight_right  hear_left  \
0      1   35    170     75      90.0         1.0         1.0         1
1      1   30    180     80      89.0         0.9         1.2         1
2      1   40    165     75      91.0         1.2         1.5         1
3      1   50    175     80      91.0         1.5         1.2         1
4      1   50    165     60      80.0         1.0         1.2         1
...
3344    1   70    165     65      86.0         1.2         1.2         1
3345    1   45    160     65      87.0         0.7         0.7         1
3346    0   60    165     60      92.0         0.7         0.9         1
3347    1   55    175     75      80.0         0.9         1.0         1
3348    1   65    160     55      83.0         0.3         0.4         1

    hear_right  SBP  ...  HDL_chole  LDL_chole  triglyceride  hemoglobin  \
0              1  120  ...         48        126           92         17.1
1              1  130  ...         55        148          121         15.8
2              1  120  ...         41         74          104         15.8
3              1  145  ...         76        104          106         17.6
4              1  138  ...         61        117          104         13.8
...
3344            1  132  ...         55        116          100         14.0
3345            1  138  ...         63         71           96         16.1
3346            1  138  ...         62         98          140         14.8
3347            1  131  ...         51        138           59         15.0
3348            1  110  ...         58         84           78         13.9

    urine_protein  serum_creatinine  SGOT_AST  SGOT_ALT  gamma_GTP  \
0                1                1.0        21        35         40
1                1                0.9        20        36         27
2                1                0.9        47        32         68
3                1                1.1        29        34         18
4                1                0.8        19        12         25
...
3344            1                0.9        23        21         16
3345            1                0.9        33        25         68
3346            1                0.6        24        19          8

```

3347	1	0.8	15	31	33
3348	1	0.6	20	19	21

	SMK_stat_type_cd
0	1
1	3
2	1
3	1
4	1
...	...
3344	2
3345	1
3346	1
3347	2
3348	3

[3349 rows x 23 columns]

```
[162]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(features, target,
↳test_size=0.2, random_state=42)
```

2 Perceptron Before PCA

```
[163]: from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
model=Perceptron()
model.fit(X_train ,y_train)
y_pred=model.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print("Accuracy:",accuracy)
```

Accuracy: 0.7059701492537314

```
[164]: from sklearn.metrics import confusion_matrix
c=confusion_matrix(y_test,y_pred)
c
```

```
[164]: array([[246, 95],
[102, 227]])
```

```
[165]: from sklearn.metrics import classification_report
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.71	0.72	0.71	341
1	0.70	0.69	0.70	329
accuracy			0.71	670
macro avg	0.71	0.71	0.71	670
weighted avg	0.71	0.71	0.71	670

3 Logistic Regression Before PCA

```
[166]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
model=LogisticRegression()
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print("Accuracy:",accuracy)
```

Accuracy: 0.7223880597014926

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
[167]: from sklearn.metrics import confusion_matrix
c=confusion_matrix(y_test,y_pred)
c
```

```
[167]: array([[246, 95],
[ 91, 238]])
```

```
[168]: from sklearn.metrics import classification_report
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.73	0.72	0.73	341
1	0.71	0.72	0.72	329
accuracy			0.72	670
macro avg	0.72	0.72	0.72	670

weighted avg	0.72	0.72	0.72	670
--------------	------	------	------	-----

3.1 SVM Before PCA

```
[169]: from sklearn.svm import SVC
model=SVC()
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print("Accuracy:",accuracy)
```

Accuracy: 0.7059701492537314

```
[170]: from sklearn.metrics import confusion_matrix
c=confusion_matrix(y_test,y_pred)
c
```

```
[170]: array([[245,  96],
        [101, 228]])
```

```
[171]: from sklearn.metrics import classification_report
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.71	0.72	0.71	341
1	0.70	0.69	0.70	329
accuracy			0.71	670
macro avg	0.71	0.71	0.71	670
weighted avg	0.71	0.71	0.71	670

3.2 KNN Before PCA

```
[172]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
y_pred = knn_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("KNN Accuracy:", accuracy)
```

KNN Accuracy: 0.6283582089552239

4 PCA

```
[173]: from sklearn.decomposition import PCA
pca = PCA(n_components=6)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

5 KNN After PCA

```
[174]: from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
knn_model_pca = KNeighborsClassifier(n_neighbors=5)
knn_model_pca.fit(X_train_pca, y_train)
y_pred_pca = knn_model_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, y_pred_pca)
print("KNN Accuracy after PCA:", accuracy_pca)
```

KNN Accuracy after PCA: 0.5761194029850746

6 SVM After PCA

```
[175]: from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
model = SVC()
model.fit(X_train_pca, y_train)
y_pred = model.predict(X_test_pca)
accuracy = accuracy_score(y_test, y_pred)
print("SVC Accuracy with PCA:", accuracy)
print(classification_report(y_test, y_pred))
```

SVC Accuracy with PCA: 0.6268656716417911

	precision	recall	f1-score	support
0	0.61	0.76	0.68	341
1	0.66	0.49	0.56	329
accuracy			0.63	670
macro avg	0.63	0.62	0.62	670
weighted avg	0.63	0.63	0.62	670

7 Logistic Regression After PCA

```
[176]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
model = LogisticRegression()
model.fit(X_train_pca, y_train)
y_pred = model.predict(X_test_pca)
accuracy = accuracy_score(y_test, y_pred)
print("Logistic Regression Accuracy with PCA:", accuracy)
print(classification_report(y_test, y_pred))
```

Logistic Regression Accuracy with PCA: 0.6104477611940299

	precision	recall	f1-score	support
0	0.59	0.75	0.66	341
1	0.64	0.47	0.54	329
accuracy			0.61	670
macro avg	0.62	0.61	0.60	670
weighted avg	0.62	0.61	0.60	670

8 Perceptron After PCA

```
[177]: from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
model = Perceptron()
model.fit(X_train_pca, y_train)
y_pred = model.predict(X_test_pca)
accuracy = accuracy_score(y_test, y_pred)
print("Perceptron Accuracy with PCA:", accuracy)
print(classification_report(y_test, y_pred))
```

Perceptron Accuracy with PCA: 0.591044776119403

	precision	recall	f1-score	support
0	0.59	0.64	0.61	341
1	0.59	0.54	0.57	329
accuracy			0.59	670
macro avg	0.59	0.59	0.59	670
weighted avg	0.59	0.59	0.59	670

9 Resampling

```
[178]: from sklearn.model_selection import train_test_split
from sklearn.utils import resample
for i in range(0, 10):
    xr, yr = resample(features, target)
    x_train, x_test, y_train, y_test = train_test_split(xr, yr, test_size=0.2)
```

9.1 Logistic Regression After PCA

```
[179]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
model = LogisticRegression()
accuracies_logistic = []
for i in range(0, 10):
    xr, yr = resample(features, target)
    x_train, x_test, y_train, y_test = train_test_split(xr, yr, test_size=0.2)
    model = LogisticRegression()
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)
    accuracy = accuracy_score(y_test, y_pred)
    accuracies_logistic.append(accuracy)

    print(f"LogisticRegression (Iteration {i+1}): {accuracy}")
```

LogisticRegression (Iteration 1): 0.7283582089552239

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:  
ConvergenceWarning: lbfgs failed to converge (status=1):  
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
```

```
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
LogisticRegression (Iteration 2): 0.7164179104477612
```

```
LogisticRegression (Iteration 3): 0.7044776119402985
```

```
LogisticRegression (Iteration 4): 0.7059701492537314
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
```

```
ConvergenceWarning: lbfgs failed to converge (status=1):
```

```
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
```

```
ConvergenceWarning: lbfgs failed to converge (status=1):
```

```
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
LogisticRegression (Iteration 5): 0.7134328358208956
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
```

```
ConvergenceWarning: lbfgs failed to converge (status=1):
```

```
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
```

```
ConvergenceWarning: lbfgs failed to converge (status=1):
```

```
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(
LogisticRegression (Iteration 6): 0.744776119402985
LogisticRegression (Iteration 7): 0.7149253731343284
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(
LogisticRegression (Iteration 8): 0.7089552238805971
LogisticRegression (Iteration 9): 0.7149253731343284
LogisticRegression (Iteration 10): 0.7313432835820896
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

10 Perceptron After PCA

```
[180]: from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
accuracies_perceptron = []
for i in range(0, 10):
    xr, yr = resample(features, target)
    x_train, x_test, y_train, y_test = train_test_split(xr, yr, test_size=0.2)
    model = Perceptron()
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)
    accuracy = accuracy_score(y_test, y_pred)
    accuracies_perceptron.append(accuracy)
    print(f"Perceptron Accuracy (Iteration {i+1}): {accuracy}")
```

```
Perceptron Accuracy (Iteration 1): 0.6417910447761194
Perceptron Accuracy (Iteration 2): 0.5582089552238806
Perceptron Accuracy (Iteration 3): 0.6492537313432836
Perceptron Accuracy (Iteration 4): 0.6582089552238806
Perceptron Accuracy (Iteration 5): 0.5134328358208955
Perceptron Accuracy (Iteration 6): 0.5582089552238806
Perceptron Accuracy (Iteration 7): 0.5791044776119403
Perceptron Accuracy (Iteration 8): 0.5925373134328358
Perceptron Accuracy (Iteration 9): 0.5388059701492537
Perceptron Accuracy (Iteration 10): 0.48059701492537316
```

11 KNN After PCA

```
[181]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
accuracies_knn = []
for i in range(0, 10):
    xr, yr = resample(features, target)
    x_train, x_test, y_train, y_test = train_test_split(xr, yr, test_size=0.2)
    model = KNeighborsClassifier()
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)
    accuracy = accuracy_score(y_test, y_pred)
    accuracies_knn.append(accuracy)
    print(f"KNN Accuracy (Iteration {i+1}): {accuracy}")
```

```
KNN Accuracy (Iteration 1): 0.6761194029850747
KNN Accuracy (Iteration 2): 0.6850746268656717
KNN Accuracy (Iteration 3): 0.6895522388059702
KNN Accuracy (Iteration 4): 0.7417910447761195
KNN Accuracy (Iteration 5): 0.6701492537313433
KNN Accuracy (Iteration 6): 0.7
```

```
KNN Accuracy (Iteration 7): 0.7194029850746269
KNN Accuracy (Iteration 8): 0.7223880597014926
KNN Accuracy (Iteration 9): 0.6985074626865672
KNN Accuracy (Iteration 10): 0.7313432835820896
```

12 SVM After PCA

```
[182]: from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
accuracies_svc = []
for i in range(0, 10):
    xr, yr = resample(features, target)
    x_train, x_test, y_train, y_test = train_test_split(xr, yr, test_size=0.2)
    model = SVC()
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)
    accuracy = accuracy_score(y_test, y_pred)
    accuracies_svc.append(accuracy)
    print(f"SVC Accuracy (Iteration {i+1}): {accuracy}")
```

```
SVC Accuracy (Iteration 1): 0.746268656716418
SVC Accuracy (Iteration 2): 0.7029850746268657
SVC Accuracy (Iteration 3): 0.7149253731343284
SVC Accuracy (Iteration 4): 0.6955223880597015
SVC Accuracy (Iteration 5): 0.7164179104477612
SVC Accuracy (Iteration 6): 0.735820895522388
SVC Accuracy (Iteration 7): 0.7208955223880597
SVC Accuracy (Iteration 8): 0.6940298507462687
SVC Accuracy (Iteration 9): 0.7014925373134329
SVC Accuracy (Iteration 10): 0.7134328358208956
```

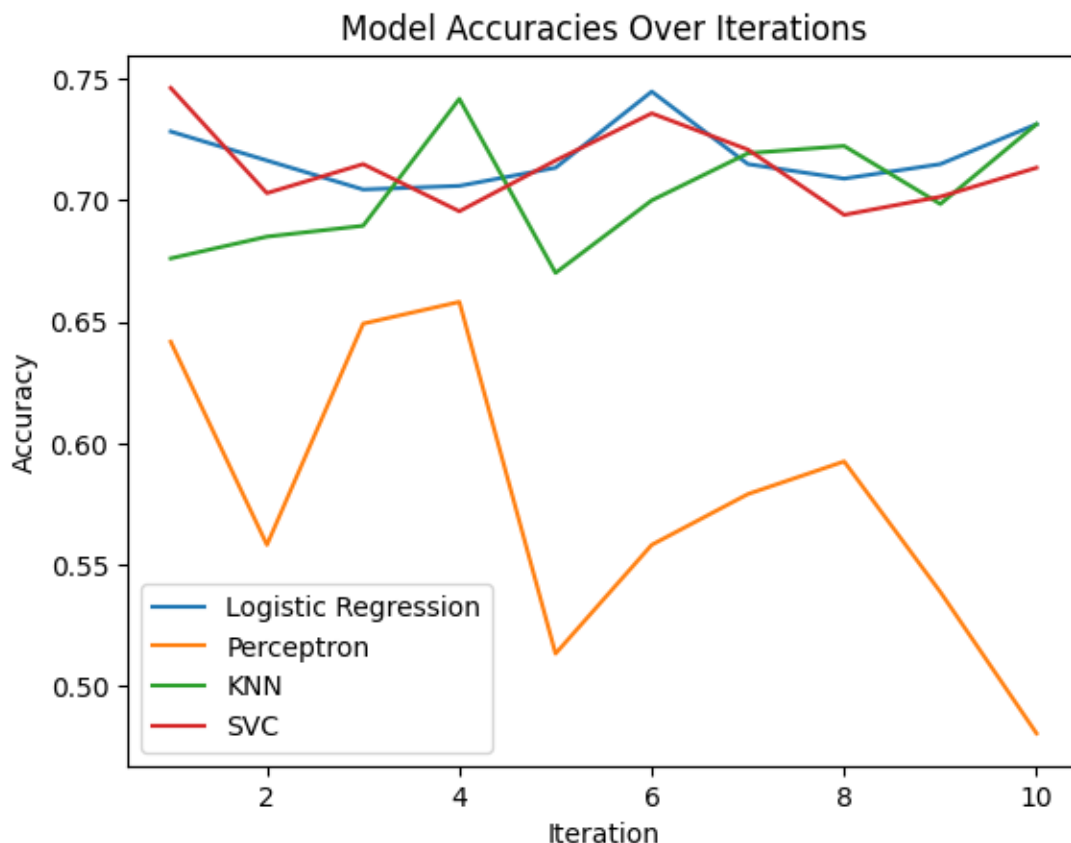
13 Plotting Graphs Of All Models

```
[183]: import matplotlib.pyplot as plt
import numpy as np
a=np.arange(1,11)
print(a)
print(accuracies_logistic)
print(accuracies_perceptron)
print(accuracies_knn)
print(accuracies_svc)
plt.plot(range(1, 11), accuracies_logistic, label='Logistic Regression')
plt.plot(range(1, 11), accuracies_perceptron, label='Perceptron')
plt.plot(range(1, 11), accuracies_knn, label='KNN')
plt.plot(range(1, 11), accuracies_svc, label='SVC')
plt.xlabel('Iteration')
```



```
plt.ylabel('Accuracy')
plt.title('Model Accuracies Over Iterations')
plt.legend()
plt.show()
```

```
[ 1  2  3  4  5  6  7  8  9 10]
[0.7283582089552239, 0.7164179104477612, 0.7044776119402985, 0.7059701492537314,
0.7134328358208956, 0.744776119402985, 0.7149253731343284, 0.7089552238805971,
0.7149253731343284, 0.7313432835820896]
[0.6417910447761194, 0.5582089552238806, 0.6492537313432836, 0.6582089552238806,
0.5134328358208955, 0.5582089552238806, 0.5791044776119403, 0.5925373134328358,
0.5388059701492537, 0.48059701492537316]
[0.6761194029850747, 0.6850746268656717, 0.6895522388059702, 0.7417910447761195,
0.6701492537313433, 0.7, 0.7194029850746269, 0.7223880597014926,
0.6985074626865672, 0.7313432835820896]
[0.746268656716418, 0.7029850746268657, 0.7149253731343284, 0.6955223880597015,
0.7164179104477612, 0.735820895522388, 0.7208955223880597, 0.6940298507462687,
0.7014925373134329, 0.7134328358208956]
```



14 Finding min & max in Logistic Regression

```
[184]: min_accuracy_logistic = min(accuracies_logistic)
max_accuracy_logistic = max(accuracies_logistic)

print(f"Minimum Logistic Regression Accuracy: {min_accuracy_logistic}")
print(f"Maximum Logistic Regression Accuracy: {max_accuracy_logistic}")
print("difference: ", max_accuracy_logistic - min_accuracy_logistic)
```

Minimum Logistic Regression Accuracy: 0.7044776119402985
Maximum Logistic Regression Accuracy: 0.744776119402985
difference: 0.040298507462686595

15 Finding min & max in Perceptron

```
[185]: min_accuracy_perceptron = min(accuracies_perceptron)
max_accuracy_perceptron = max(accuracies_perceptron)

print(f"Minimum Perceptron Accuracy: {min_accuracy_perceptron}")
print(f"Maximum Perceptron Accuracy: {max_accuracy_perceptron}")
print("difference: ", max_accuracy_perceptron - min_accuracy_perceptron)
```

Minimum Perceptron Accuracy: 0.48059701492537316
Maximum Perceptron Accuracy: 0.6582089552238806
difference: 0.1776119402985074

16 Finding min & max in KNN

```
[186]: min_accuracy_knn = min(accuracies_knn)
max_accuracy_knn = max(accuracies_knn)

print(f"Minimum KNN Accuracy: {min_accuracy_knn}")
print(f"Maximum KNN Accuracy: {max_accuracy_knn}")
print("KNN difference: ", max_accuracy_knn - min_accuracy_knn)
```

Minimum KNN Accuracy: 0.6701492537313433
Maximum KNN Accuracy: 0.7417910447761195
KNN difference: 0.07164179104477619

17 Finding min & max in SVM

```
[187]: min_accuracy_svc = min(accuracies_svc)
max_accuracy_svc = max(accuracies_svc)

print(f"Minimum SVC Accuracy: {min_accuracy_svc}")
```

```
print(f"Maximum SVC Accuracy: {max_accuracy_svc}")  
print("SVC difference: ", max_accuracy_svc - min_accuracy_svc)
```

Minimum SVC Accuracy: 0.6940298507462687

Maximum SVC Accuracy: 0.746268656716418

SVC difference: 0.052238805970149294

[187]: